

Lab03_Mikhael

Leonardo Mikhael - 266209

Atrasos de vôos

Considere novamente o problema de atrasos de vôos, disponível em <https://www.kaggle.com/usdot/flight-delays>. Nesta atividade, além dos dados de flights.csv, nós iremos utilizar informações disponíveis nos arquivos airlines.csv e airports.csv.

1. Importe, utilizando o pacote 'readr', cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados 'flights', 'airlines' e 'airports'.
 - Para o arquivo de aeroportos, importe apenas as colunas 'IATA_CODE', 'CITY', 'STATE', 'LATITUDE', 'LONGITUDE';
 - Para o arquivo de vôos:
 - a. Importe apenas as colunas 'DESTINATION_AIRPORT' e 'ARRIVAL_DELAY';
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;
 - e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 - f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(stringr)

airlines <- read_csv(
  "airlines.csv"
)
```

Rows: 14 Columns: 2

```
-- Column specification -----
Delimiter: ","
chr (2): IATA_CODE, AIRLINE

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
#a
airports <- read_csv(
  "airports.csv",
  col_types = cols_only(
    IATA_CODE = col_character(),
    CITY = col_character(),
    STATE = col_character(),
    LATITUDE = col_double(),
    LONGITUDE = col_double()
  )
)

getStats <- function(input, pos) {

  input %>%
    filter(
      !str_starts(DESTINATION_AIRPORT, "I"),
      !is.na(DESTINATION_AIRPORT),
      !is.na(ARRIVAL_DELAY)
    ) %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
```

```

      soma = sum(ARRIVAL_DELAY),
      n = n()
    )
  }

computeStats <- function(stats) {

  stats %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma = sum(soma),
      n = sum(n)
    ) %>%
    mutate(
      media_atraso = soma / n
    )
  }

mycols <- cols_only(
  DESTINATION_AIRPORT = col_character(),
  ARRIVAL_DELAY = col_double()
)

flights <- read_csv_chunked("flights.csv",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 1e6,
  col_types = mycols
)

flights <- flights %>%
  computeStats()

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas 'CITY' e 'STATE' do objeto 'airports'. Para executar esta tarefa:
 - a. Identifique a coluna que é a chave na tabela 'flights';
 - b. Identifique a coluna que é a chave na tabela 'airports';
 - c. Quais são os aeroportos que estão listados em 'flights', mas estão ausentes em 'airports'?
 - d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
 - e. Armazene a tabela resultante no objeto 'flights'.

```
#a
# A coluna 'DESTINATION_AIRPORT' é a chave na tabela 'flights'

#b
#A coluna 'IATA_CODE' é a chave na tabela 'airports'

#c
flights %>%
  anti_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) %>%
  select(DESTINATION_AIRPORT)
```

```
# A tibble: 307 x 1
  DESTINATION_AIRPORT
  <chr>
1 10135
2 10136
3 10140
4 10141
5 10146
6 10154
7 10155
8 10157
9 10158
10 10165
# i 297 more rows
```

```
#d
left_join(
  flights,
  airports %>%
    select(IATA_CODE, CITY, STATE),
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
```

```
# A tibble: 615 x 6
  DESTINATION_AIRPORT soma      n media_atraso CITY STATE
  <chr>                <dbl> <int>         <dbl> <chr> <chr>
1 10135                278    235         1.18 <NA> <NA>
```

```

2 10136          237   194          1.22 <NA> <NA>
3 10140          2954 1811          1.63 <NA> <NA>
4 10141          -134   71          -1.89 <NA> <NA>
5 10146           620   87           7.13 <NA> <NA>
6 10154           189   27           7    <NA> <NA>
7 10155          -285  141          -2.02 <NA> <NA>
8 10157          1110  118           9.41 <NA> <NA>
9 10158           880  246           3.58 <NA> <NA>
10 10165          -142    9          -15.8 <NA> <NA>
# i 605 more rows

```

```

#e
flights <- flights %>%
  left_join(
    airports %>%
      select(IATA_CODE, CITY, STATE),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )

```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```

aeroportos_estado <- airports %>%
  group_by(STATE) %>%
  summarise(
    numero_aeroportos = n()
  ) %>%
  arrange(desc(numero_aeroportos))
aeroportos_estado

```

```

# A tibble: 54 x 2
  STATE numero_aeroportos
  <chr>          <int>
1 TX              24
2 CA              22
3 AK              19
4 FL              17
5 MI              15
6 NY              14
7 CO              10
8 MN               8
9 MT               8

```

10 NC

8

```
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.
 - a. Carregue o pacote 'leaflet';
 - b. Combine os comandos 'leaflet', 'addTiles' e 'addMarkers' para a criação de um mapa básico;
 - c. Armazene o grafico em b) numa variável chamada 'this';
 - d. Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
#a
library(leaflet)

#b
mapa <- flights %>%
  left_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) %>%
  filter(
    !is.na(LATITUDE),
    !is.na(LONGITUDE)
  )

#c
this <- leaflet(mapa) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE
  )
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-31 23:55:54 -03"
```